Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería en Informática y de Sistemas
Virtualización
Sección No. 2
Ing. Guillermo Zepeda

Proyecto final

Clínica veterinaria en Kubernetes con Terraform

Ana Lucía Chávez Penagos - 1067423
Julio Javier Robles García - 1248823
Ana Sofía Asturias To - 1078123
Jorge Francisco Muñoz Portela - 1177523
Daniel Paz - 1200022
Giancarlo Ortega - 1230222

Guatemala, 10 de noviembre de 2026

Informe — Proyecto final: clínica veterinaria en Kubernetes

Curso: Virtualización, Ingeniería en Informática y de Sistemas, Universidad Rafael Landívar

Grupo: Maximos

Entrega: martes 10 de noviembre de 2026

Repositorio: https://github.com/Keinaro/proyecto-final-maximos

Imágenes: us-central1-docker.pkg.dev/veterinaria-maximos-final/vet-repo

Clúster: gke-maximos, zona us-central1-a, proyecto de Google Cloud veterinaria-maximos-final

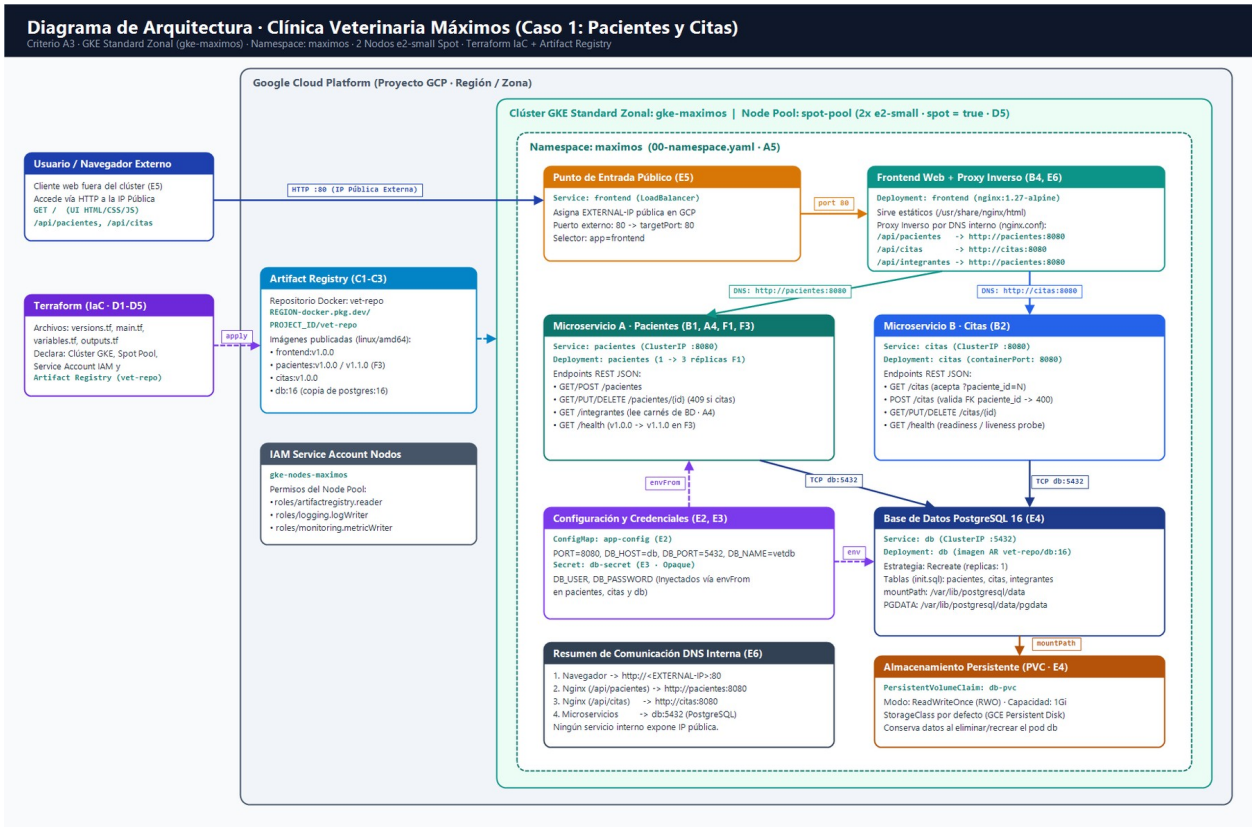
Área	Responsable	Carné
P1. Microservicio Pacientes	Giancarlo Ortega Bonilla	1230222
P2. Microservicio Citas, README y PDF	José Daniel Paz Ortega	1200022
P3. Frontend y diagrama	Ana Sofía Asturias To	1078123
P4. Base de datos y configuración	Julio Javier Robles García	1248823
P5. Terraform y costos	Ana Lucía Chávez Penagos	1067423
P6. Kubernetes, imágenes y operación	Jorge Francisco Muñoz Portela	1177523

1. Qué es

La aplicación lleva el registro de los pacientes de una clínica veterinaria (las mascotas y sus propietarios) y de las citas de cada uno. Se divide en dos microservicios, Pacientes y Citas, un frontend web y una base de datos PostgreSQL. Los microservicios están escritos en Python con FastAPI y el frontend es HTML, CSS y JavaScript servidos por nginx. Todo corre en contenedores: en la computadora con Docker Compose, y en Google Cloud sobre un clúster de Kubernetes (GKE) que se crea con Terraform.

2. Arquitectura

El navegador solo habla con el frontend, que es el único componente con IP pública (un Service LoadBalancer, 35.224.209.199). El nginx del frontend reenvía las rutas que empiezan con /api a los microservicios usando el nombre DNS de cada Service. Pacientes, Citas y la base de datos son ClusterIP, así que no se pueden alcanzar desde afuera del clúster.



Ruta pública	Destino	Puerto
/	archivos del frontend	80
/api/pacientes	pacientes	8080
/api/integrantes	pacientes	8080
/api/citas	citas	8080
/api/health/pacientes y /api/health/citas	/health de cada servicio	8080

Los dos microservicios llegan a PostgreSQL por el Service db, en el puerto 5432.

3. Datos y configuración

La base tiene tres tablas: integrantes (nombre y carné de los seis), pacientes y citas. Cada cita apunta a un paciente con una llave foránea y ON DELETE RESTRICT, así que un paciente con citas no se puede borrar: Pacientes responde 409. Si se crea una cita para un paciente que no existe, Citas responde 400. Los datos viven en un volumen persistente (PVC de 1 Gi) y siguen ahí si el Pod de la base se recrea.

La configuración que no es secreta (puerto, host y nombre de la base) está en el ConfigMap app-config. El usuario y la contraseña están en el Secret db-secret, que no se sube al repositorio. Todo está en el namespace maximos.

4. Infraestructura

Terraform crea un clúster GKE Standard de una sola zona (gke-maximos, us-central1-a), un grupo de nodos con dos máquinas e2-small Spot, el repositorio de imágenes vet-repo en Artifact Registry y la cuenta de servicio de los nodos, con permiso para leer imágenes y enviar logs y métricas. Los archivos de Terraform no tienen valores sensibles: el proyecto de Google Cloud se pasa como

variable.

5. Imágenes y despliegue

Cada servicio tiene su propio Dockerfile y su propia imagen, construida para linux/amd64 y publicada en Artifact Registry con una etiqueta de versión (v1.0.0), nunca latest. Los manifiestos de Kubernetes solo usan imágenes de Artifact Registry, por eso la imagen de PostgreSQL también se copió al repositorio. El despliegue completo se hace con scripts/deploy.sh.

Para la operación se probó escalar Pacientes a tres réplicas editando el YAML, borrar un Pod y ver cómo se recrea, y un rolling update de Pacientes de la versión v1.0.0 a la v1.1.0.

6. Costos

En la calculadora de Google Cloud la estimación fue de 108.28 USD al mes para este despliegue. El gasto real hasta el 10 de noviembre fue de 1.07 USD. Al terminar se borran los recursos de Kubernetes con scripts/deploy.sh delete y se destruye la infraestructura con terraform destroy, para que el clúster no siga cobrando.

7. Cómo ejecutarlo

En local hace falta Docker con Compose: se copia .env.example a .env, se escriben el usuario y la contraseña de la base y se corre docker compose up --build. La aplicación queda en http://localhost:8080.

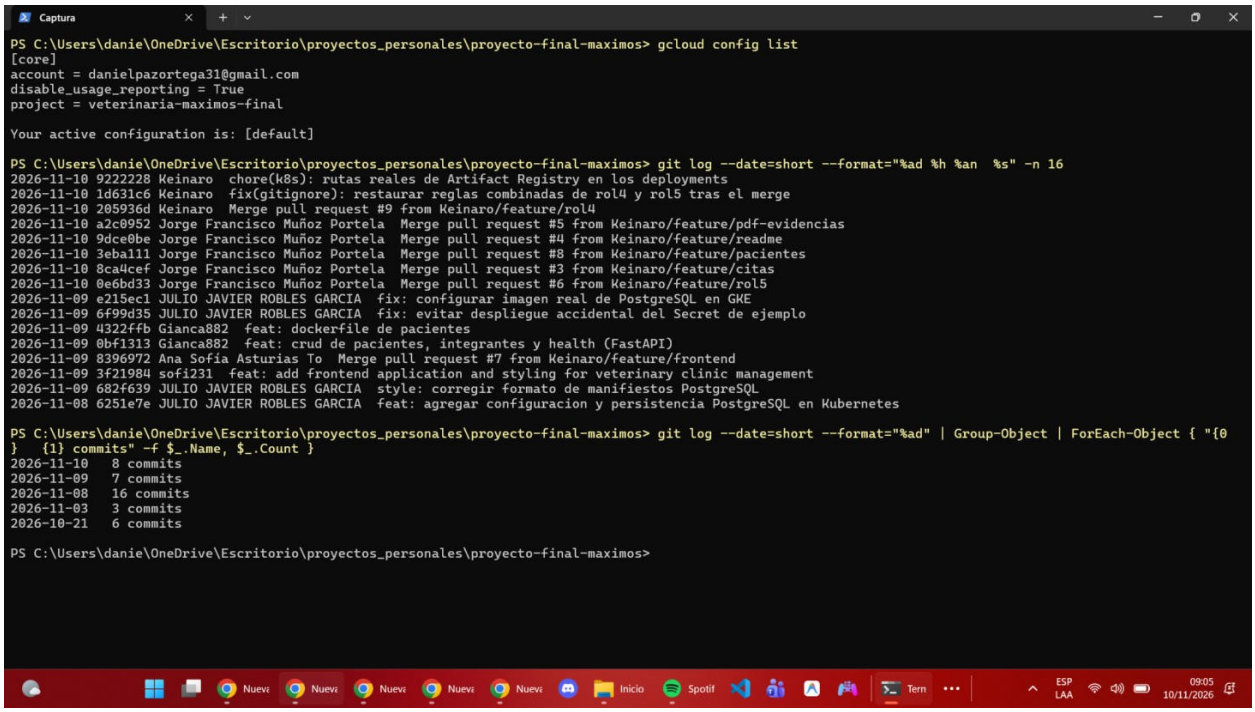
En Google Cloud se corren terraform init, plan y apply dentro de la carpeta terraform. Después se ejecutan scripts/build-push.sh y scripts/deploy.sh, y se abre en el navegador la IP pública del frontend. Los pasos completos están en el README del repositorio.

Anexo — Evidencias

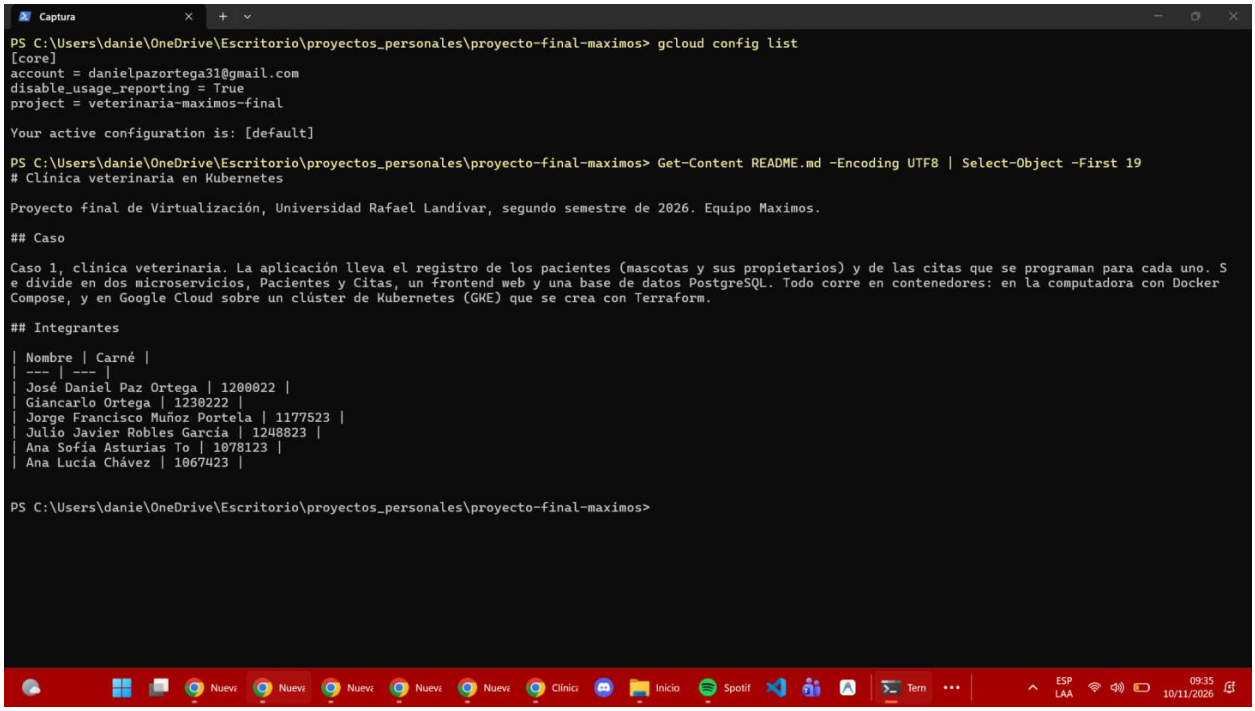
Las capturas siguen el orden de la rúbrica, de la A a la G. Entre paréntesis va quién entrega cada una (P1 a P6, según la tabla del informe). Las de la terminal del clúster muestran el reloj del sistema y la salida de gcloud config list con el proyecto del equipo. Las de Terraform y costos (D1, D2, G1, G3 y una de D5) las tomó Ana Lucía en su computadora.

A. Repositorio y documentación (14 pts)

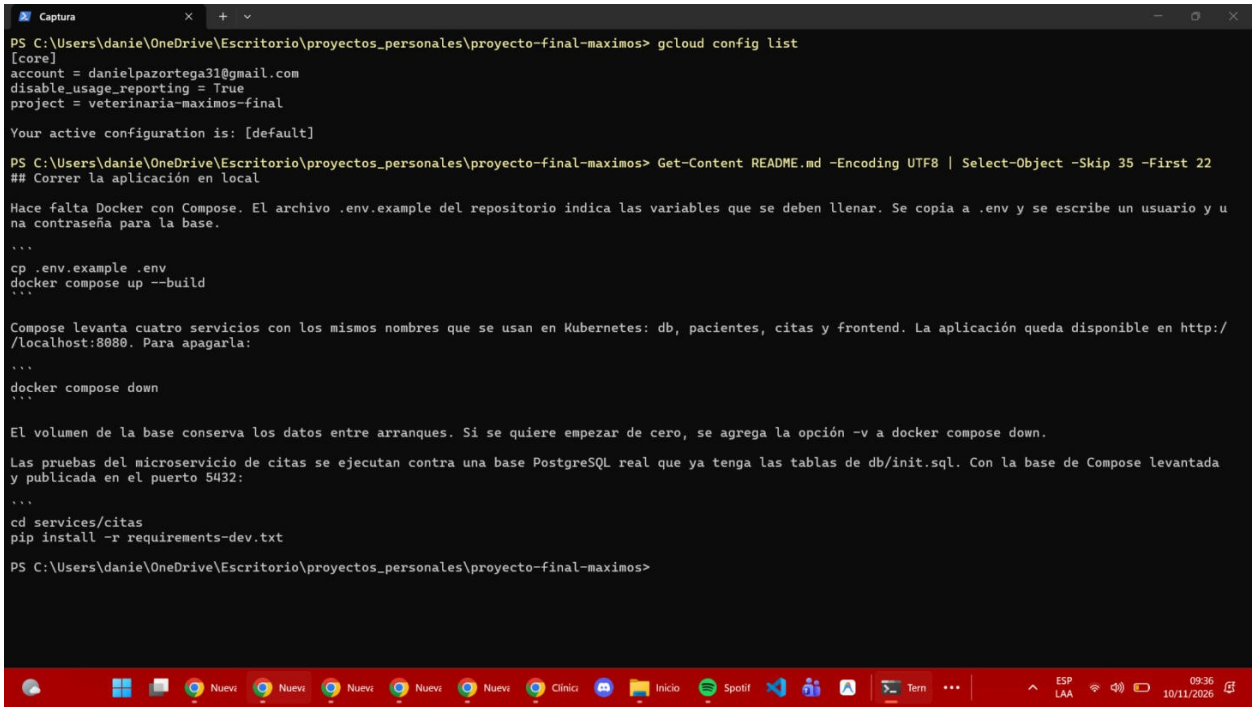
A1. Historial de commits (P1)



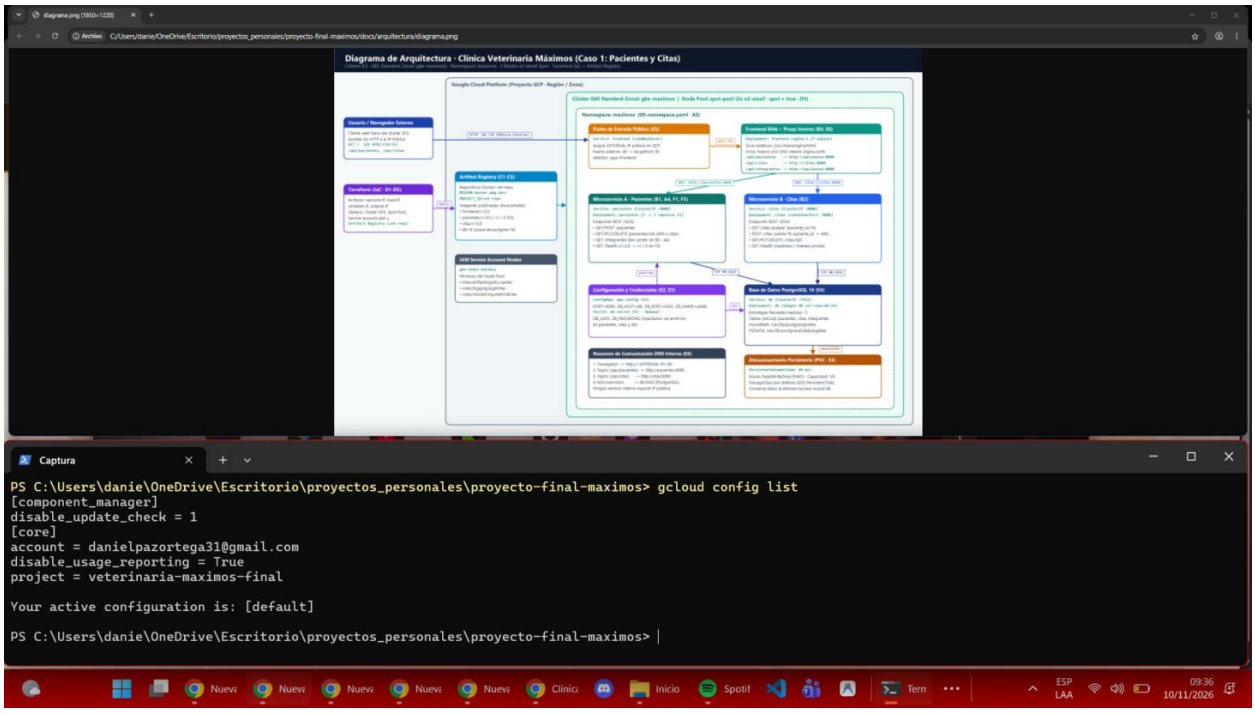
A2. README, caso e integrantes (P2)



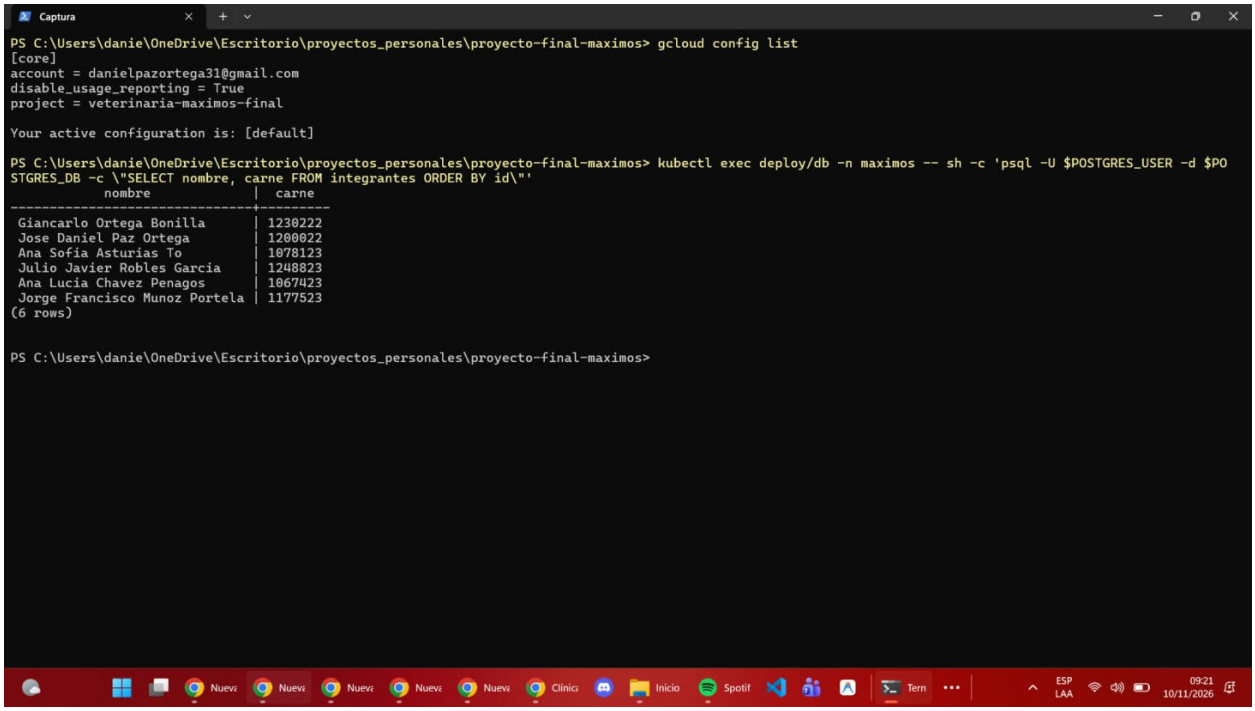
A2. README, instrucciones (P2)



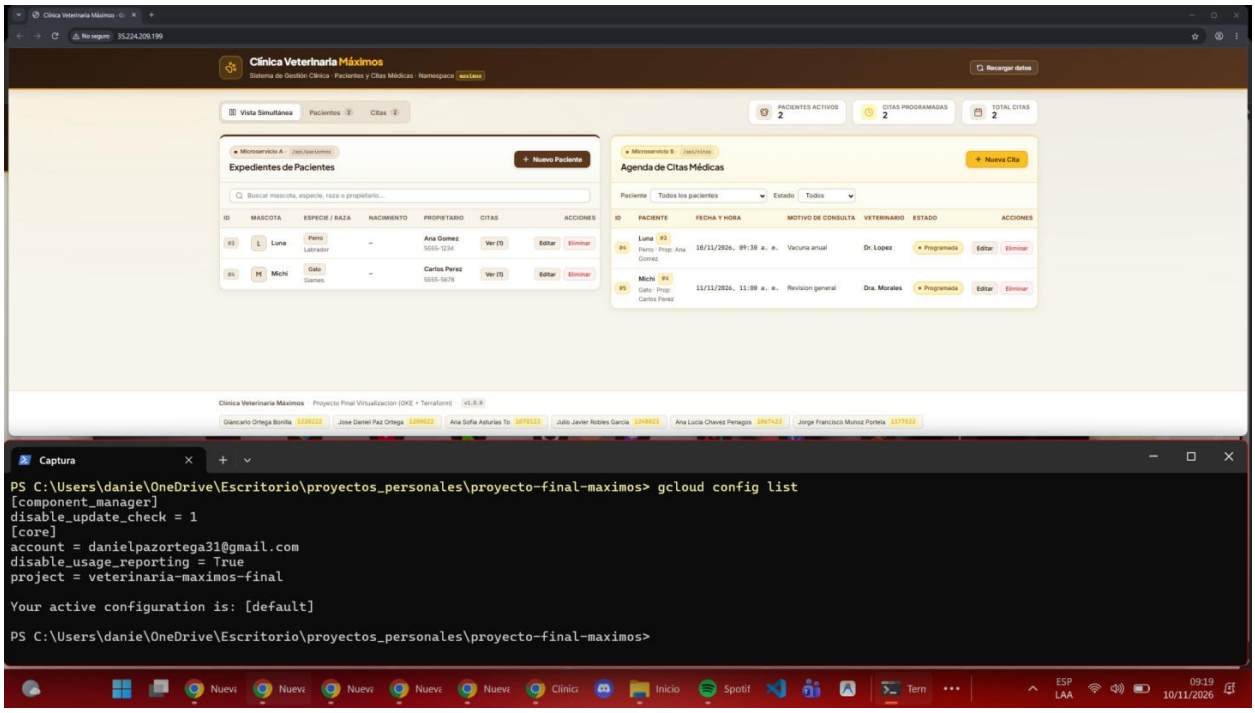
A3. Diagrama de arquitectura (P3)



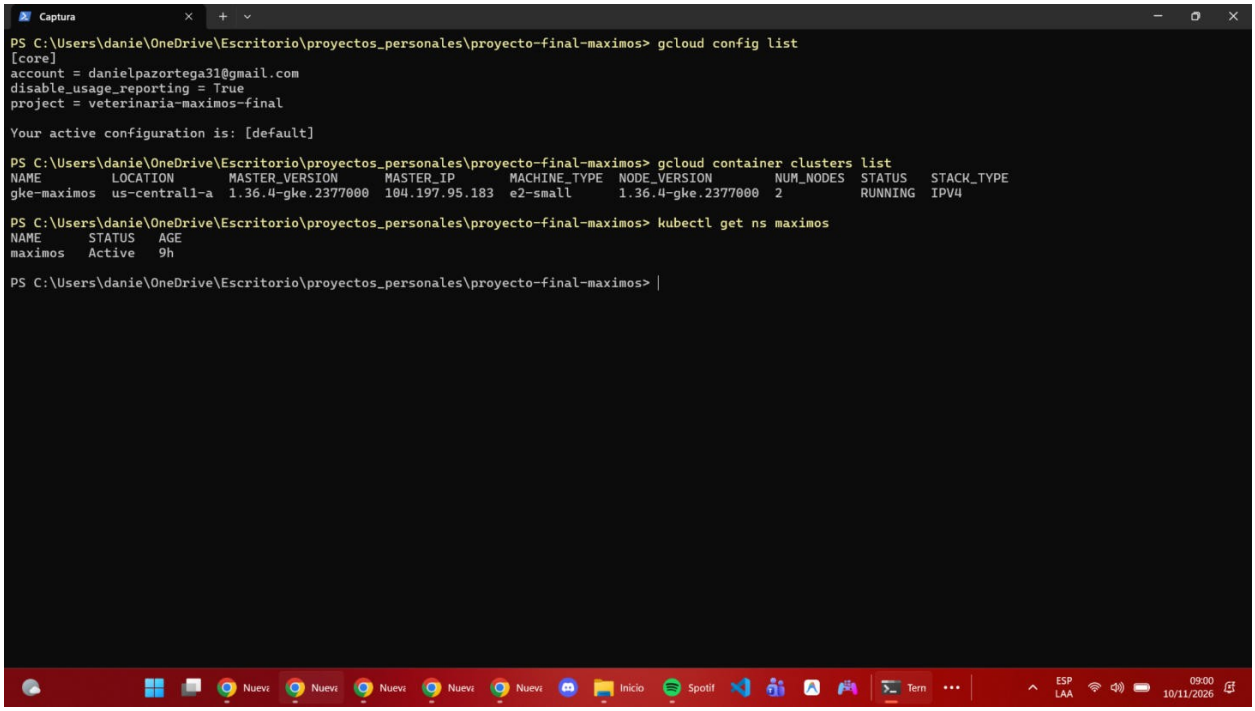
A4. Carnés en la base de datos (P4)



A4. Carnés en la interfaz (P3)

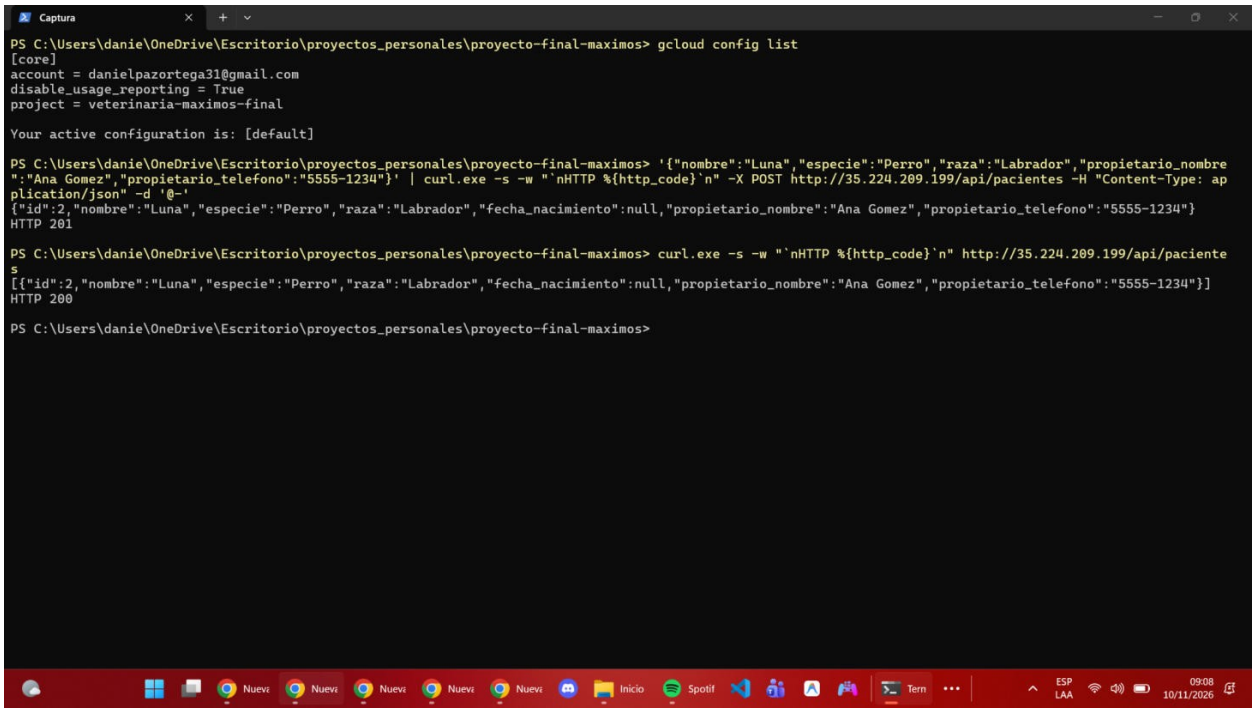


A5. Clúster y namespace (P5)

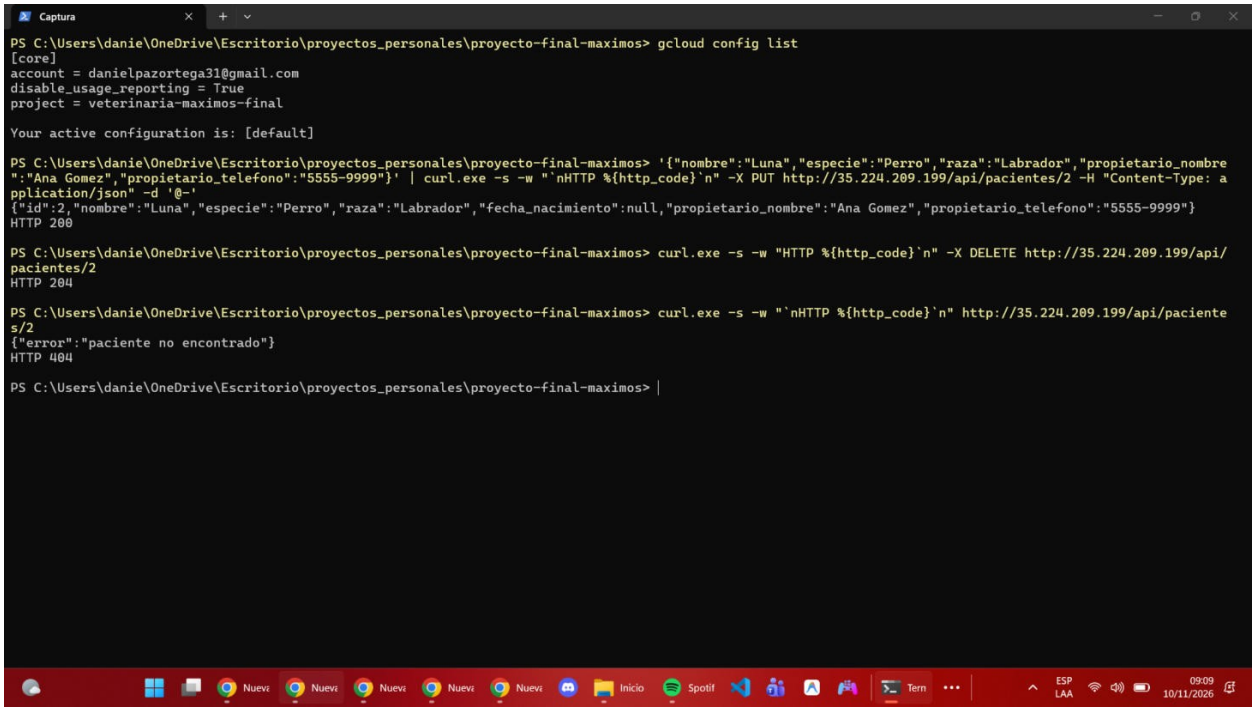


B. Microservicios (18 pts)

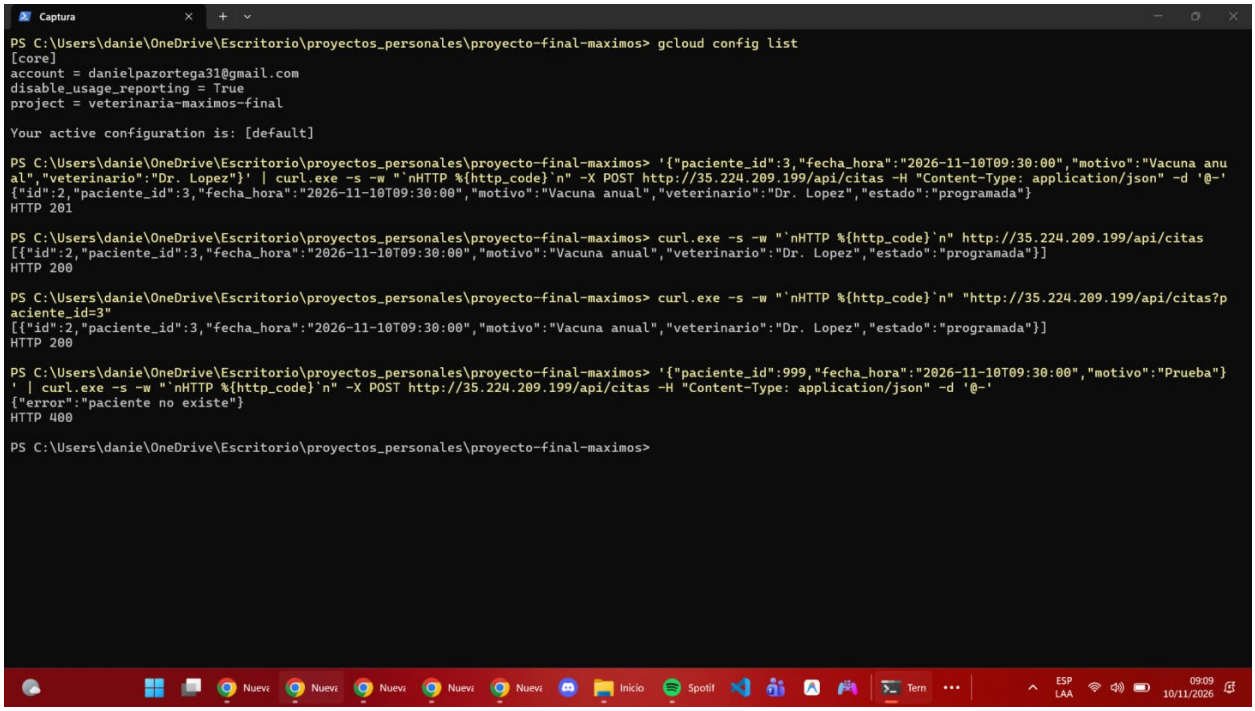
B1. Pacientes, crear y listar (P1)



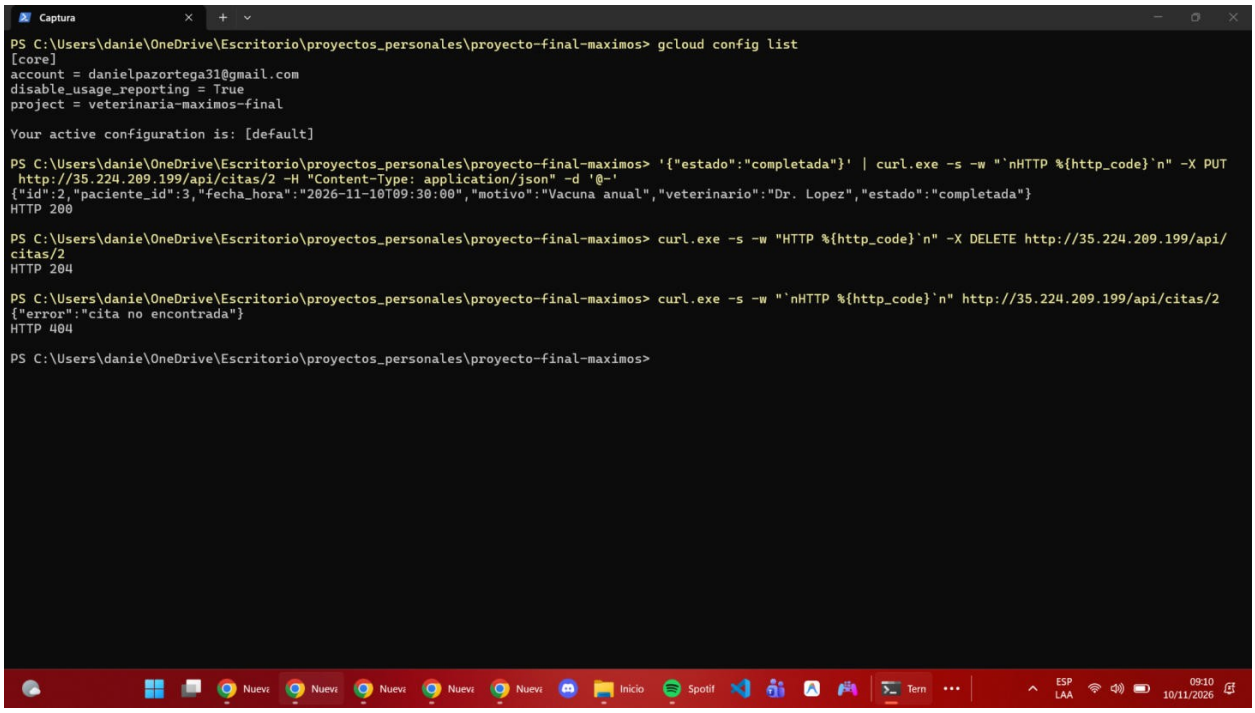
B1. Pacientes, editar y eliminar (P1)



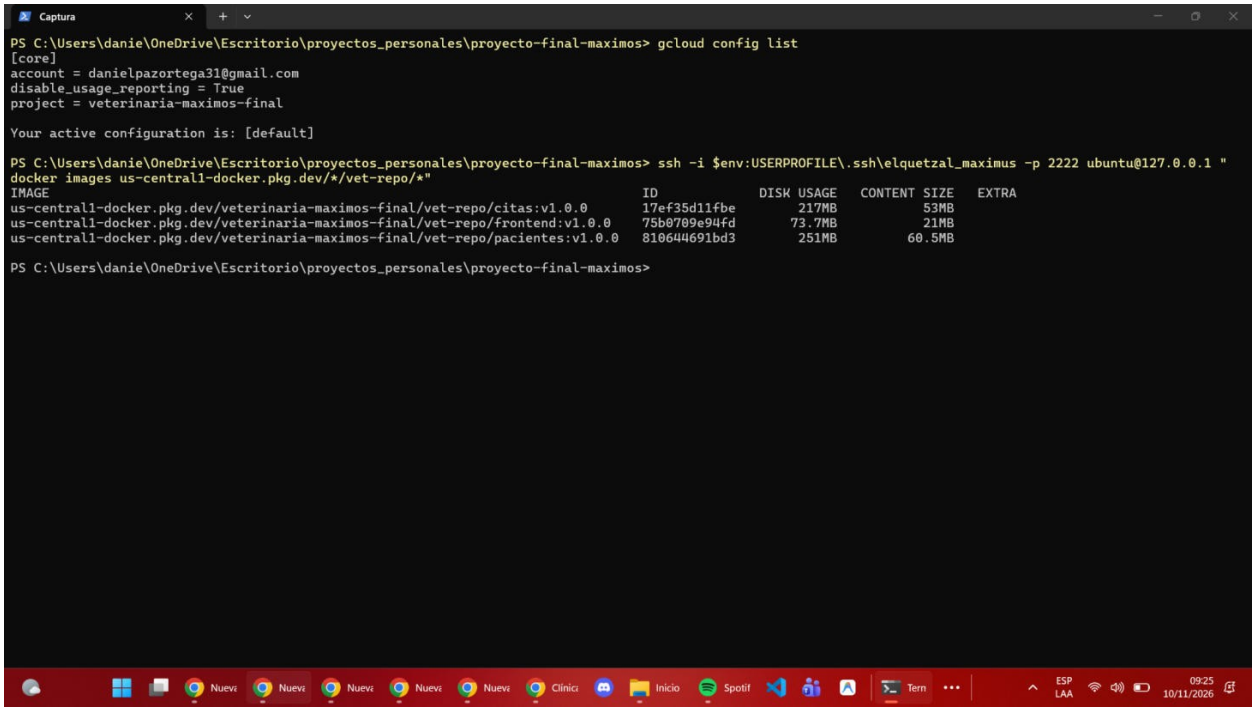
B2. Citas, crear y listar (P2)



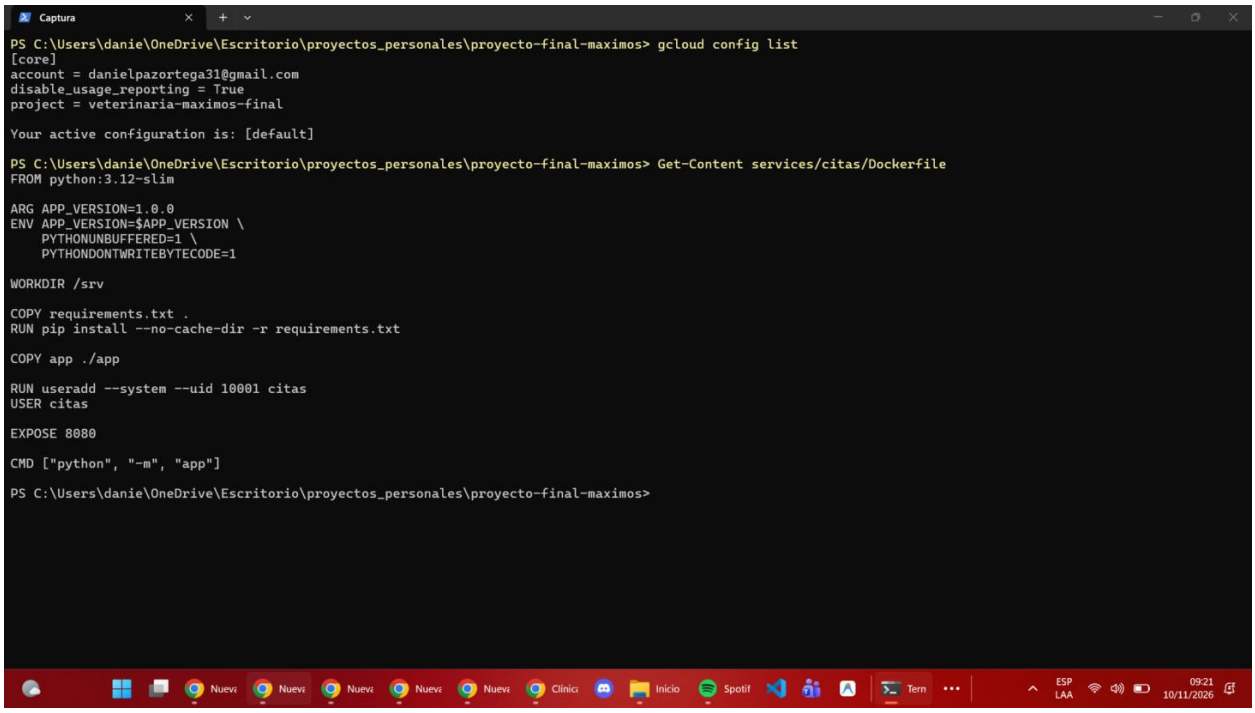
B2. Citas, editar y eliminar (P2)



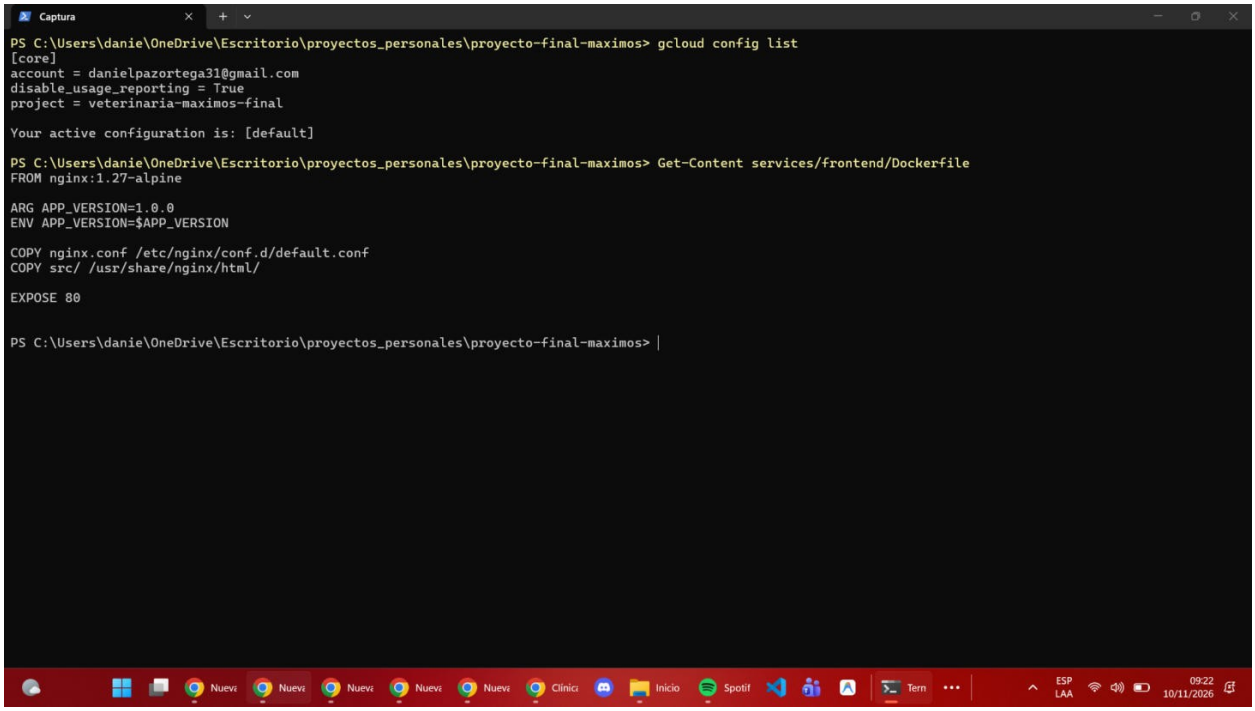
B3. docker images (P2)



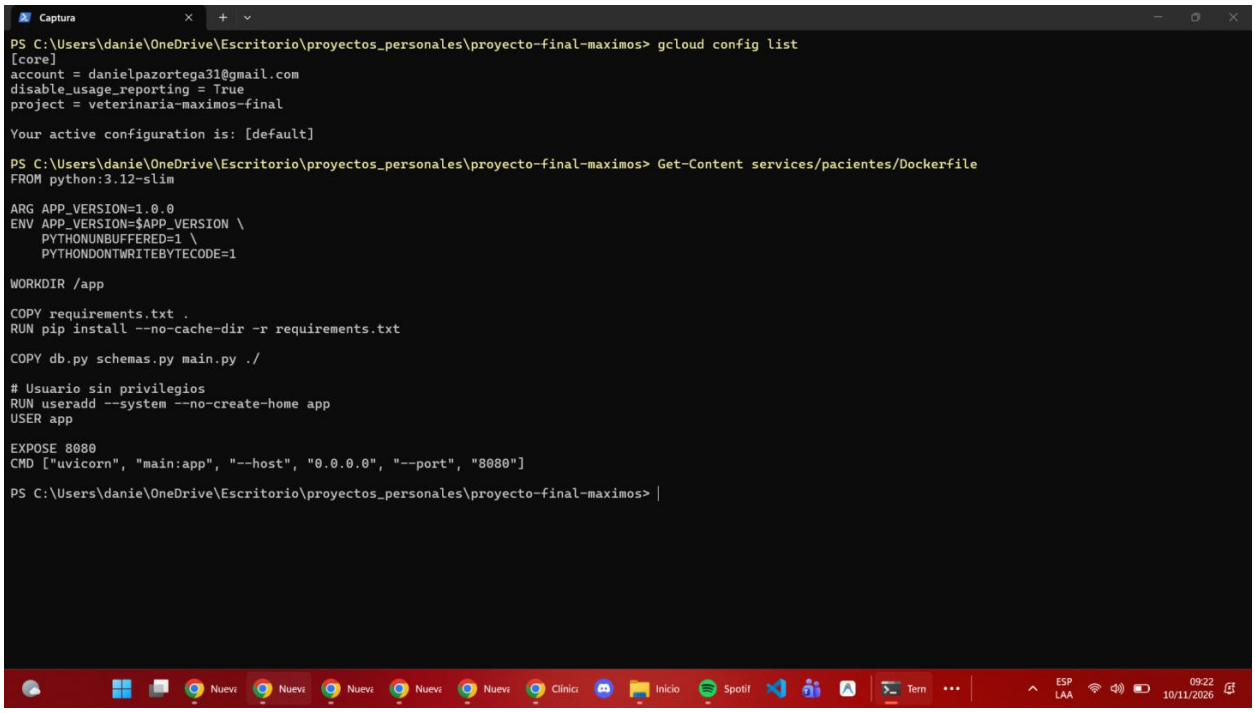
B3. Dockerfile de citas (P2)



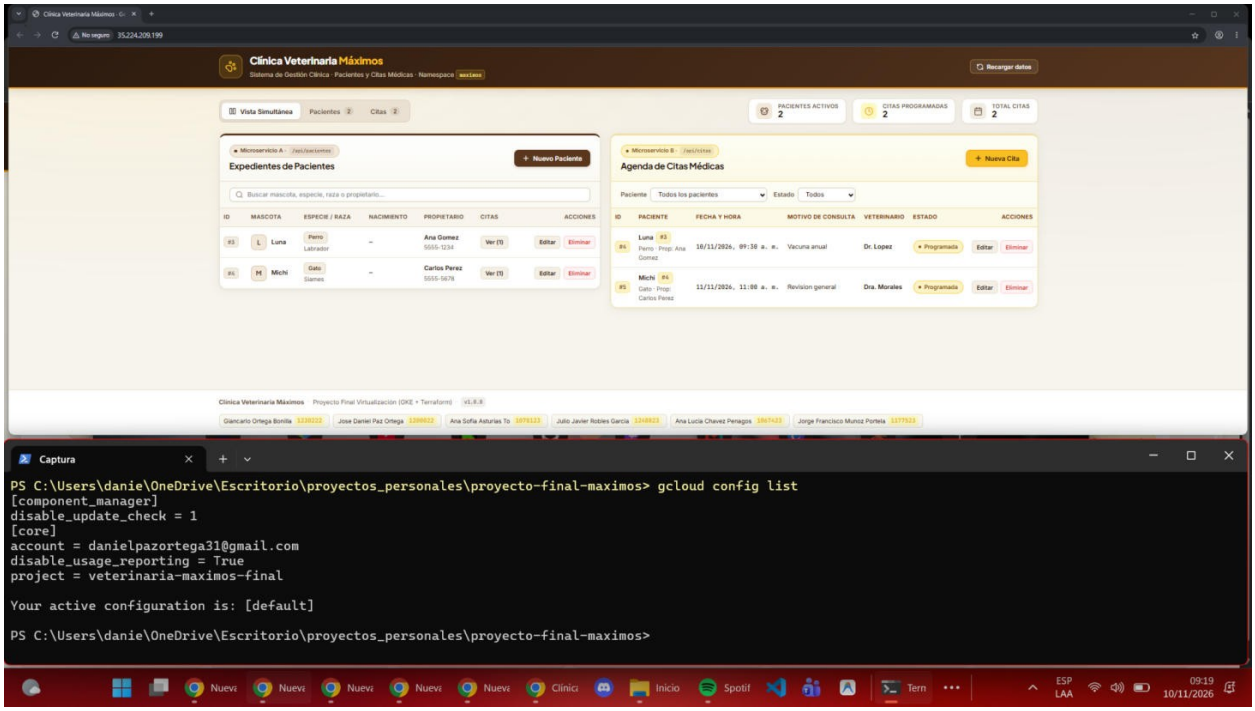
B3. Dockerfile del frontend (P3)



B3. Dockerfile de pacientes (P1)

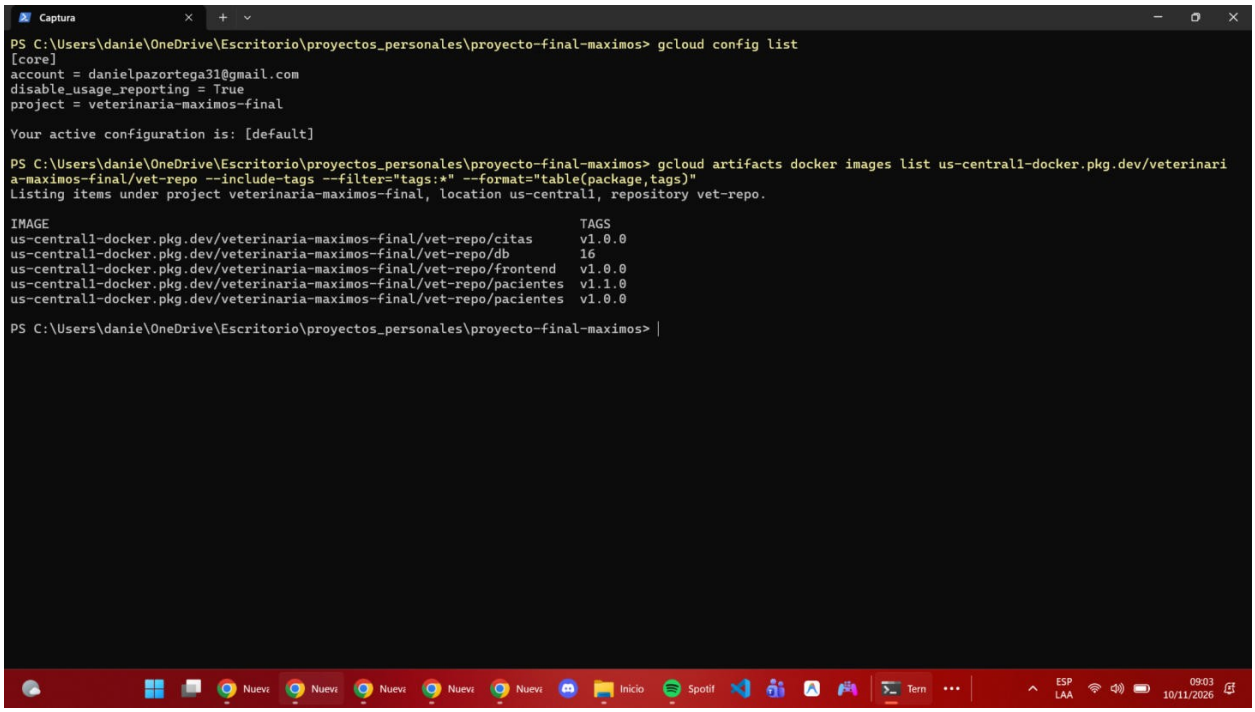


B4. Frontend con pacientes y citas (P3)

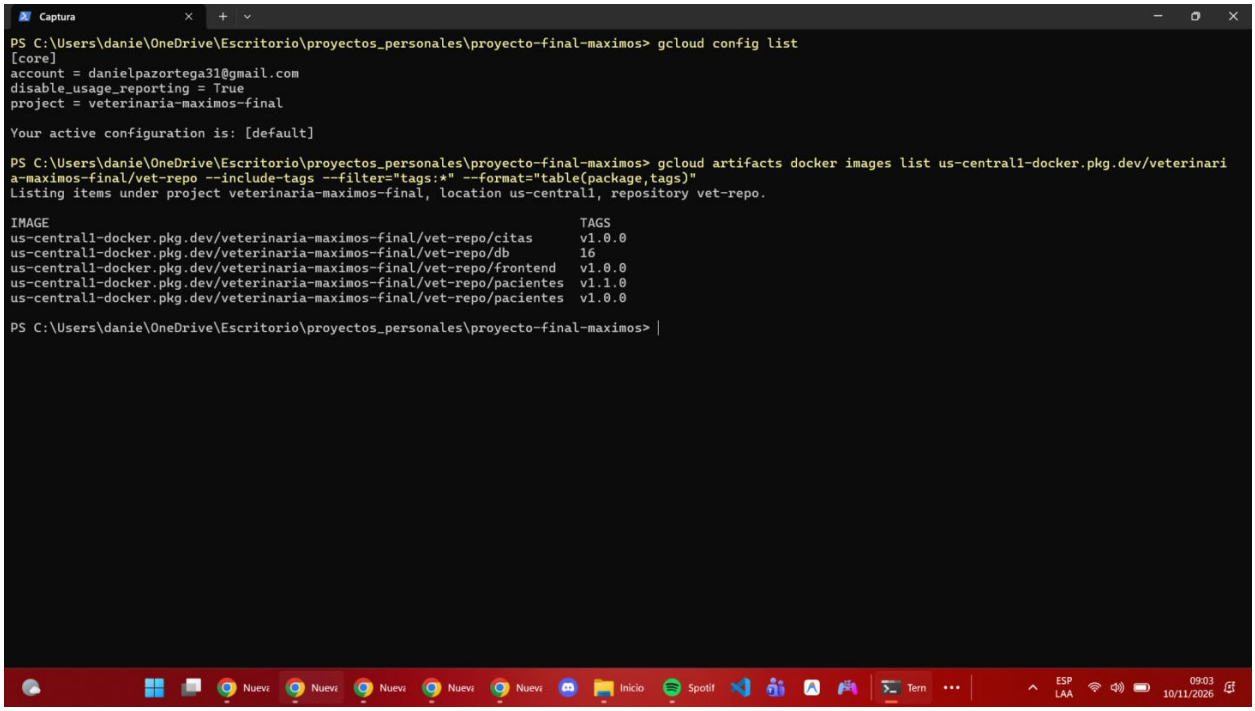


C. Imágenes y registro (10 pts)

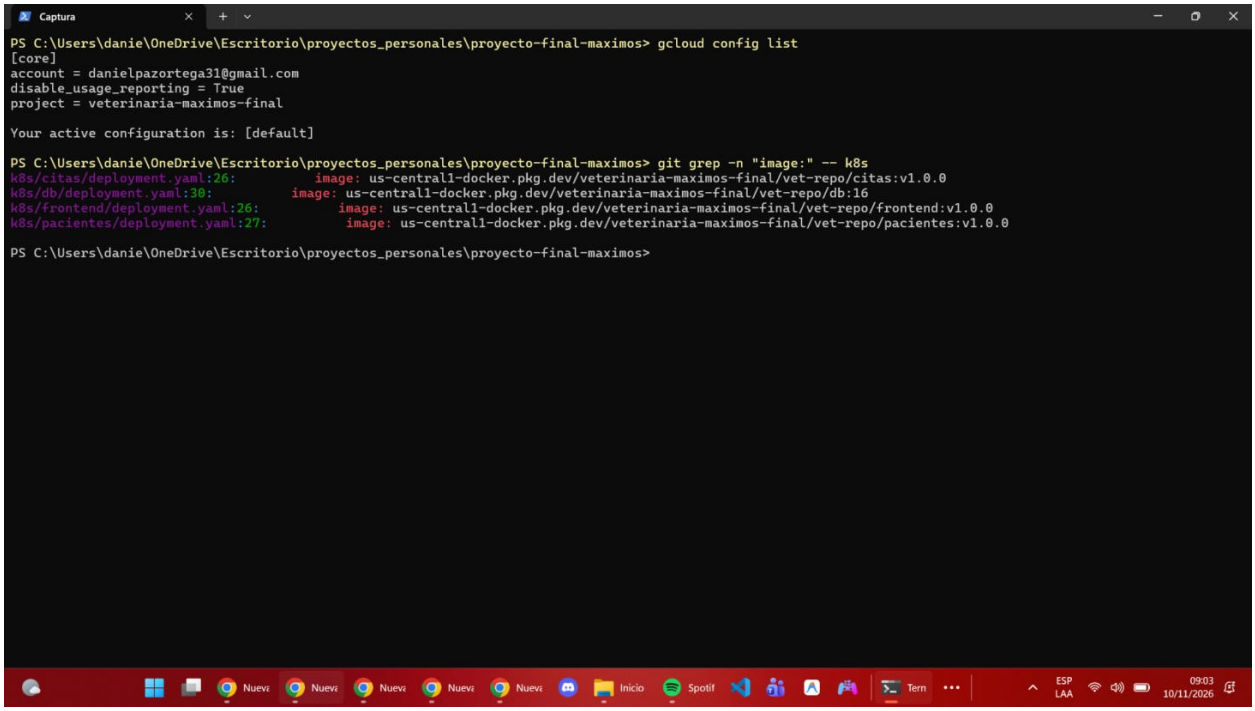
C1. Imágenes con ruta de Artifact Registry (P6)



C2. Listado del repositorio (P6)



C3. Manifiestos (P6)

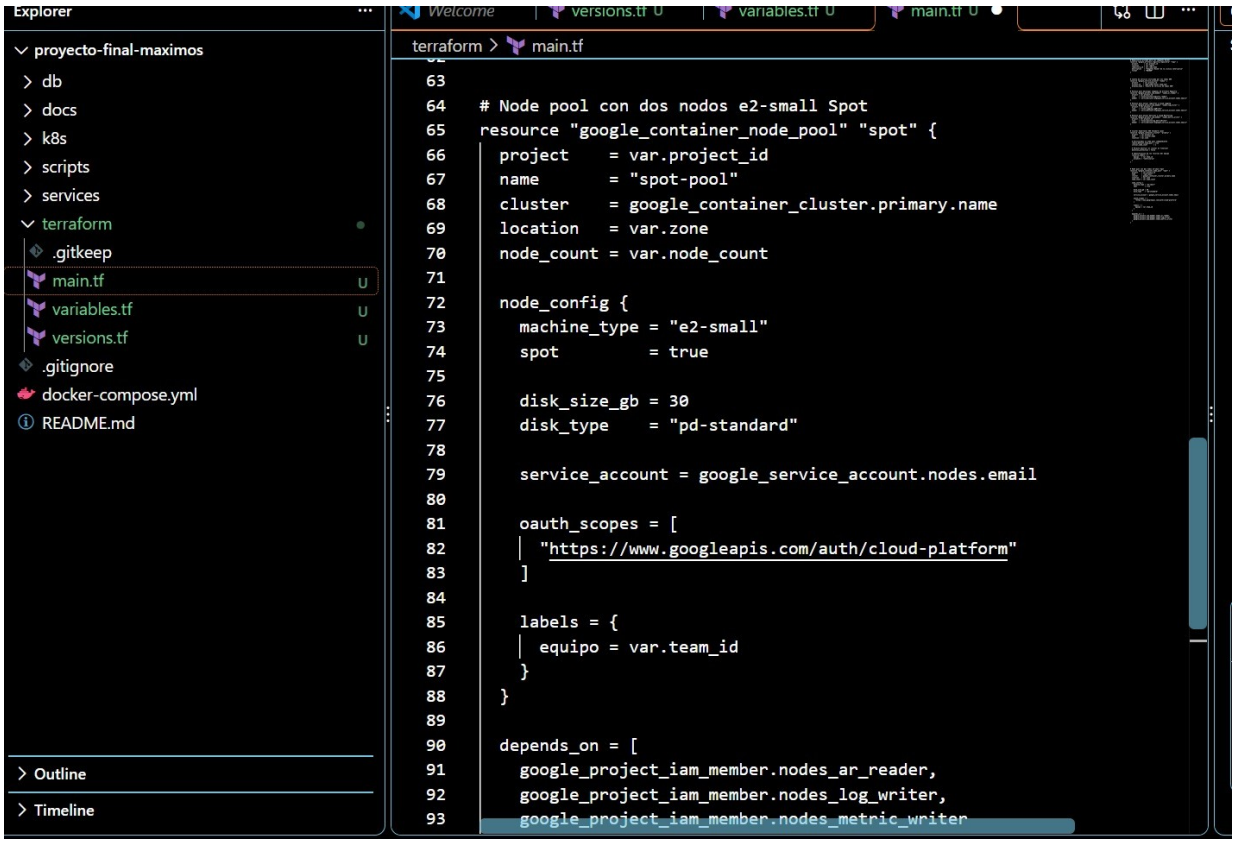


D. Infraestructura con Terraform (18 pts)

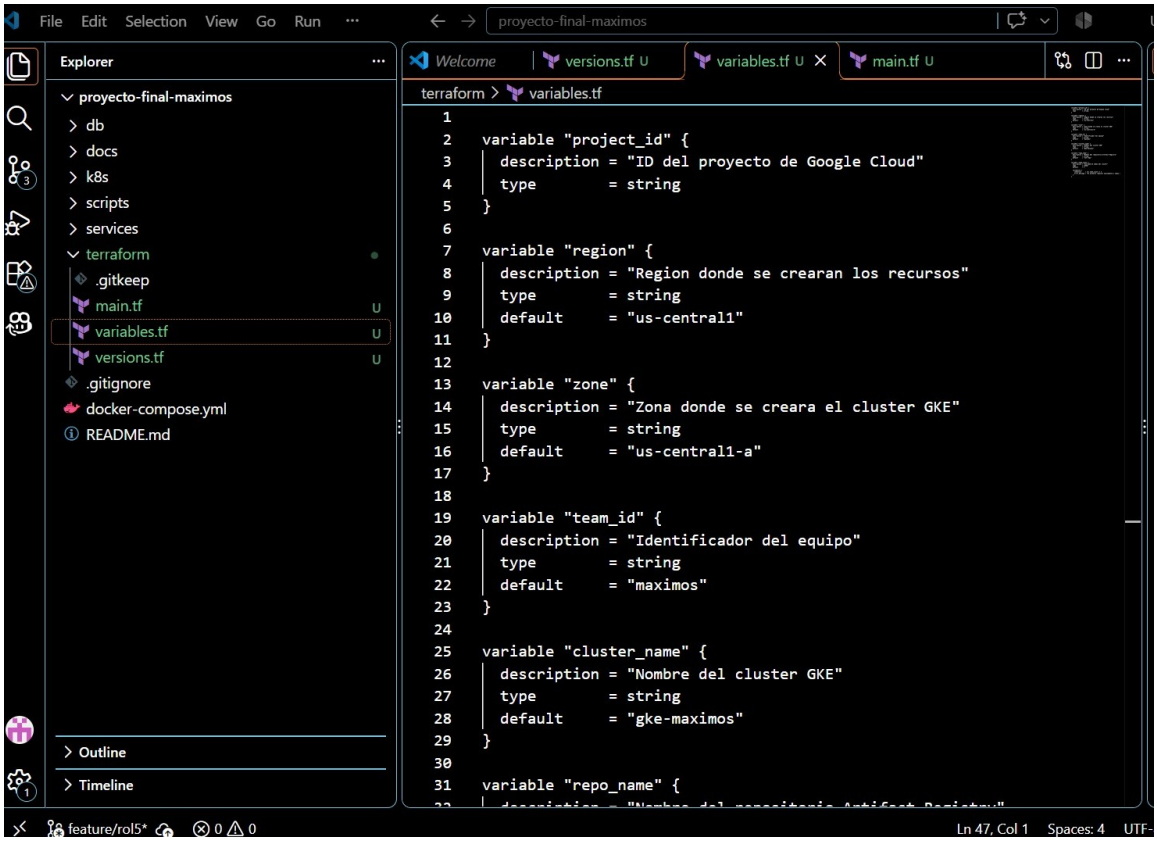
D1. Archivos de Terraform en el repositorio (P5)

```
PS C:\Users\Usuario\Downloads\VirtualizacionFinal\proyecto-final-maximos\terrafor
m> git check-ignore tfplan
tfplan
PS C:\Users\Usuario\Downloads\VirtualizacionFinal\proyecto-final-maximos\terrafor
m> git status --short
 M ../.gitignore
?? .terraform.lock.hcl
?? main.tf
?? outputs.tf
?? terraform.tfvars.example
?? variables.tf
?? versions.tf
PS C:\Users\Usuario\Downloads\VirtualizacionFinal\proyecto-final-maximos\terrafor
m>
```

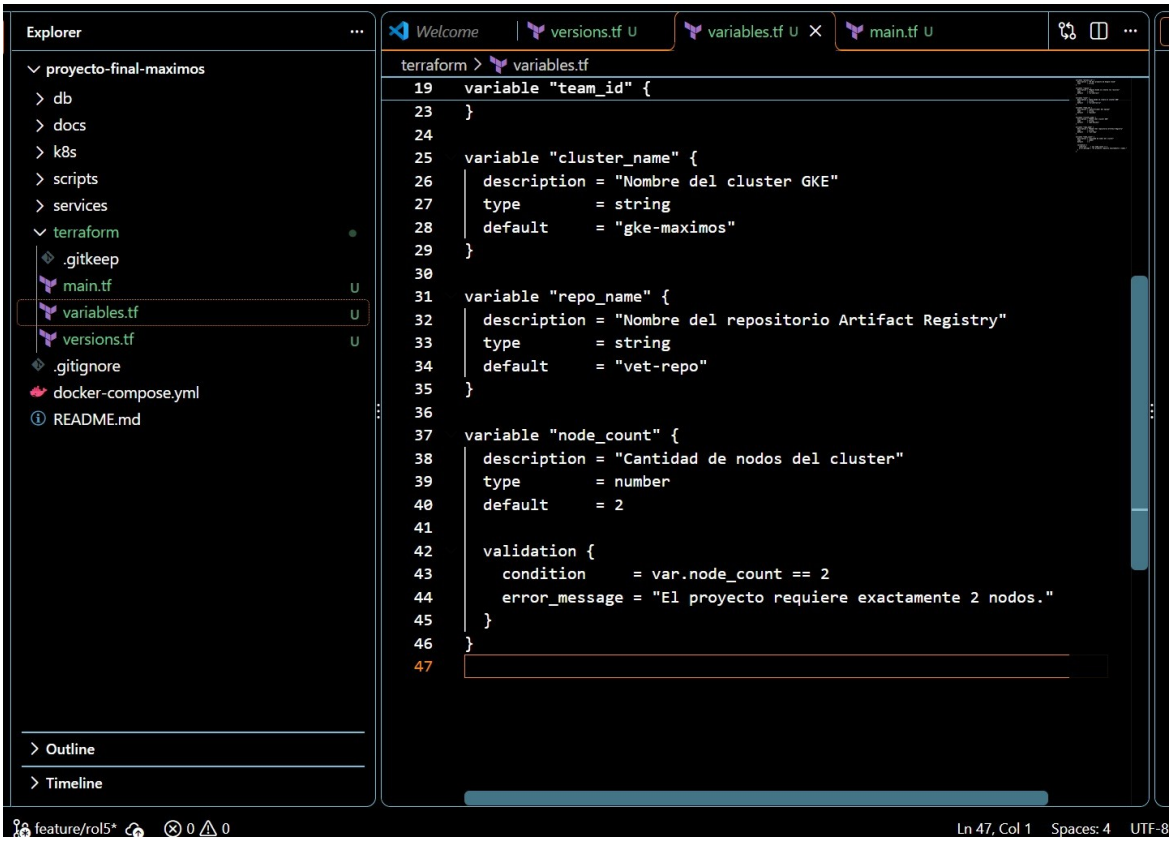
D1. main.tf, grupo de nodos Spot (P5)



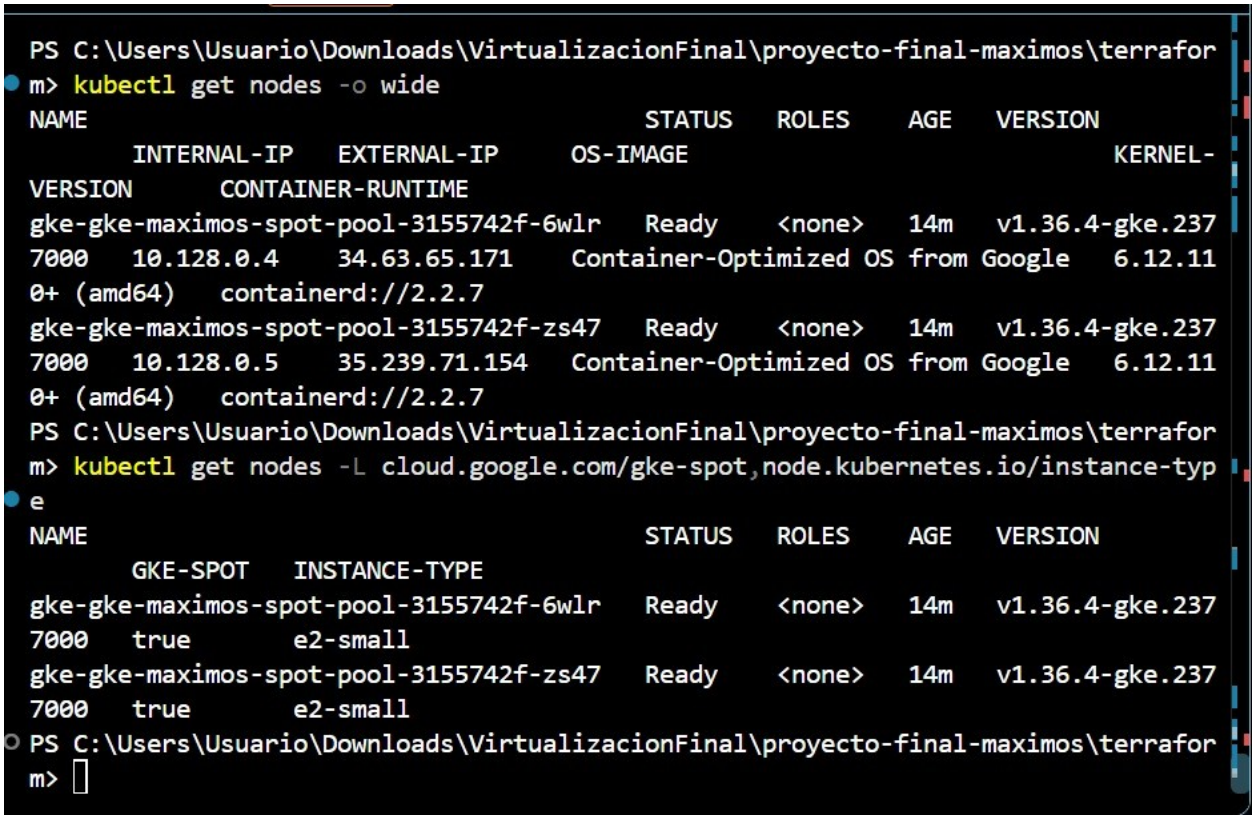
D2. variables.tf, primera parte (P5)



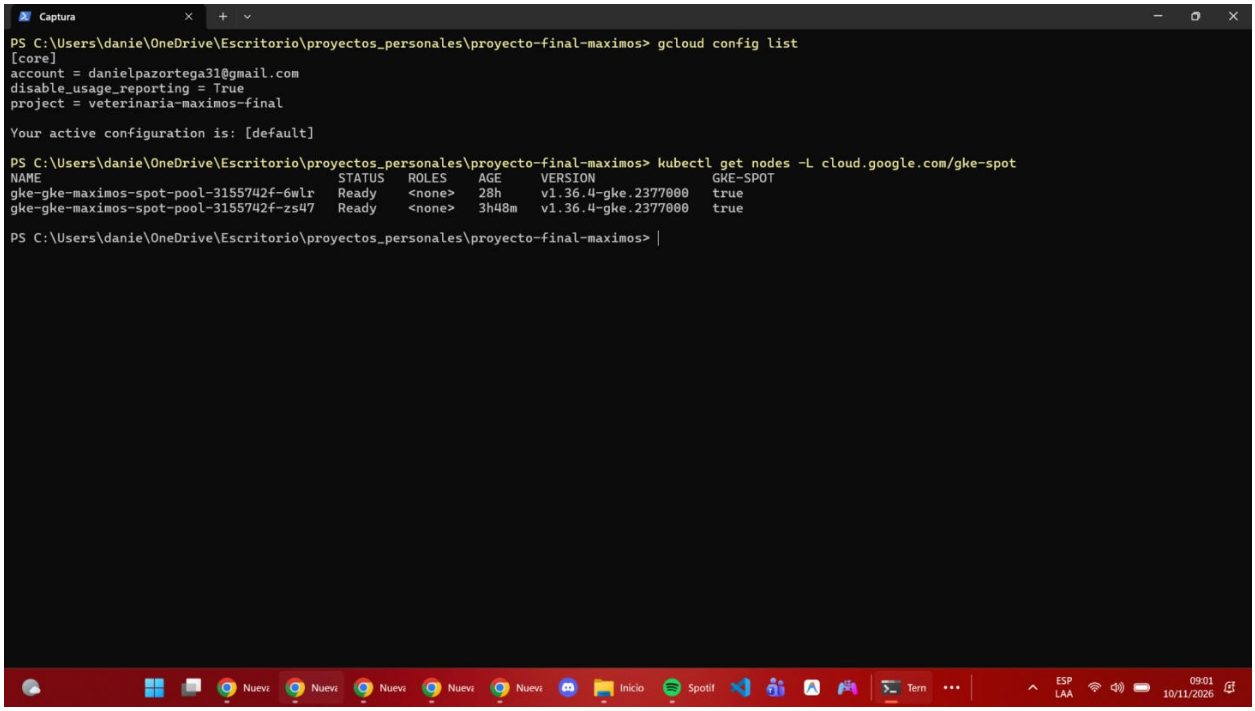
D2. variables.tf, segunda parte (P5)



D5. Nodos spot, kubectl get nodes (P5)

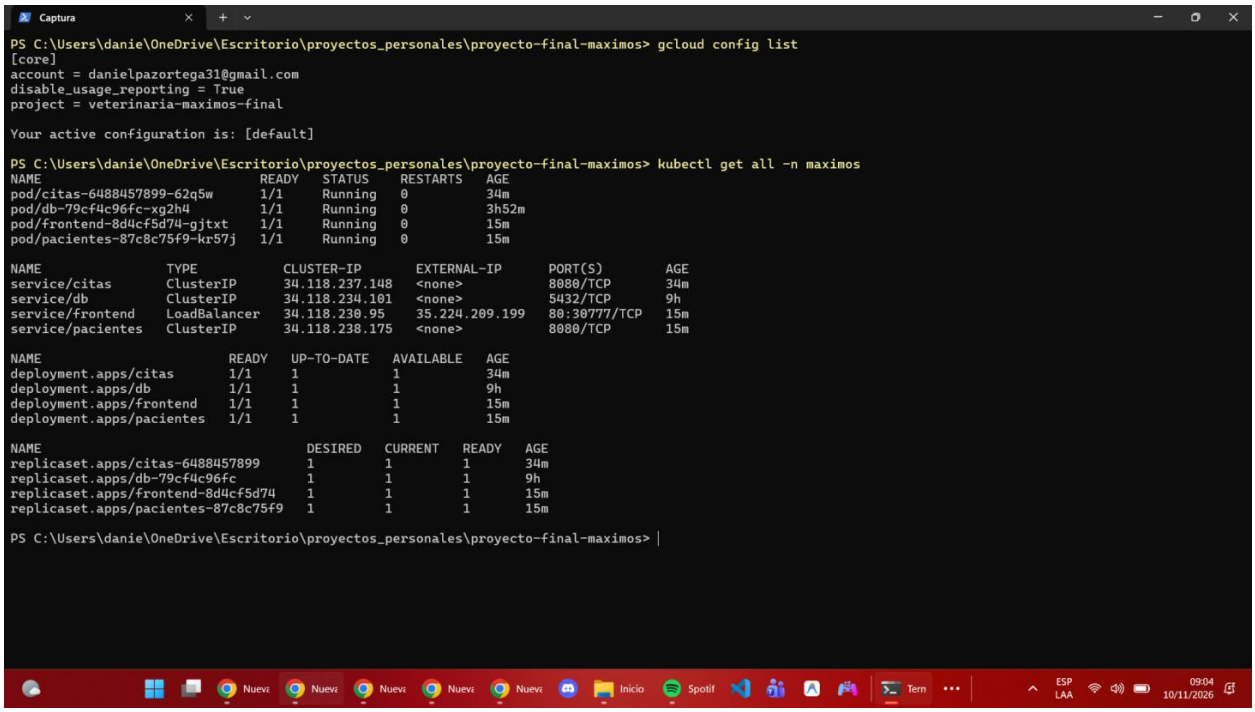


D5. Nodos spot, etiqueta gke-spot (P5)

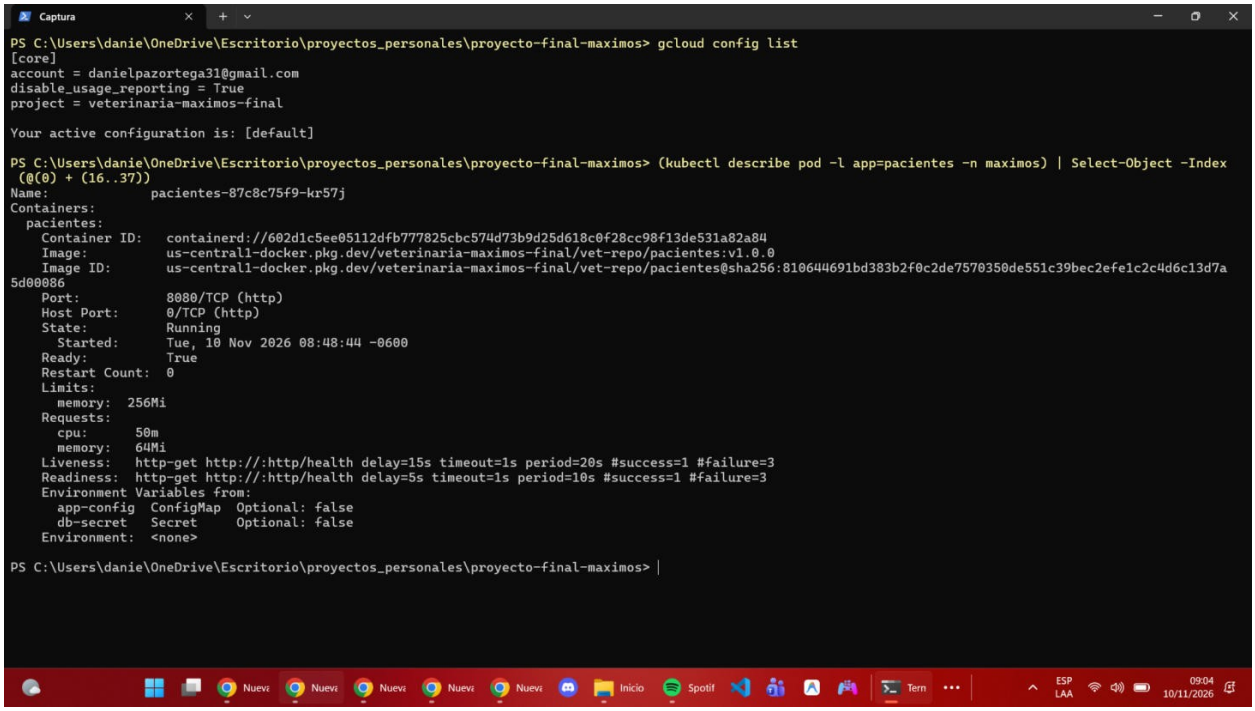


E. Despliegue en Kubernetes (22 pts)

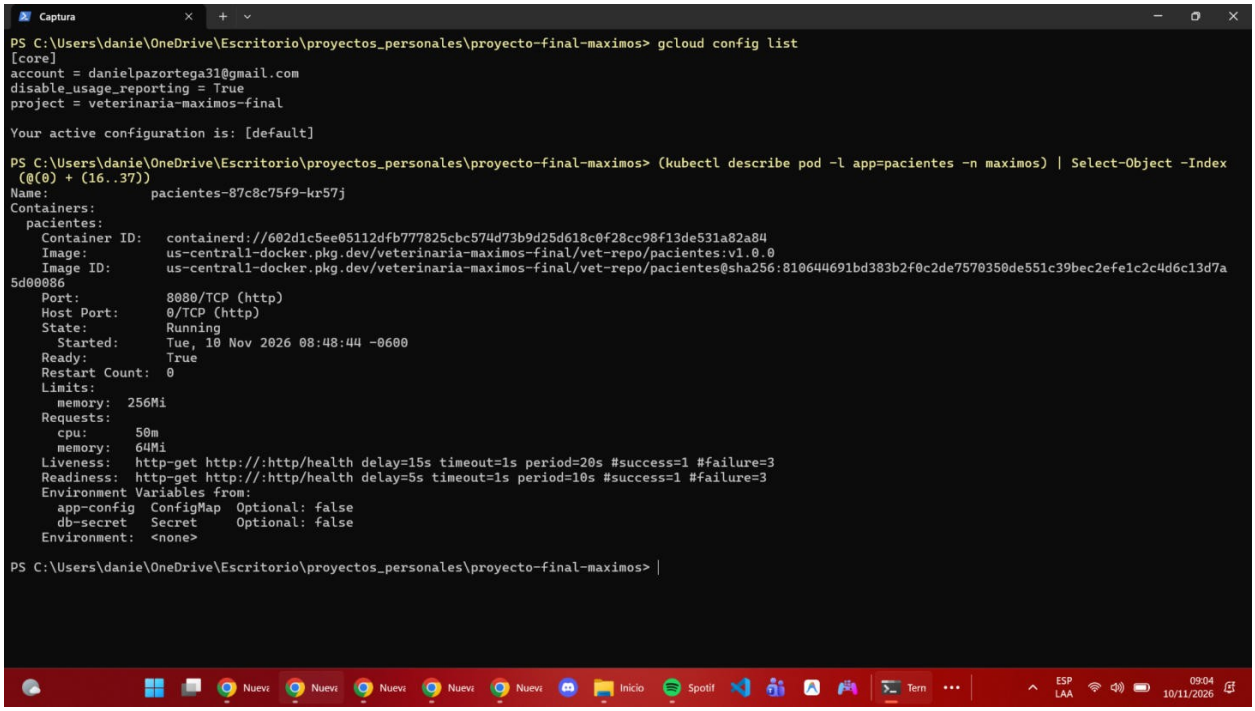
E1. kubectl get all (P6)



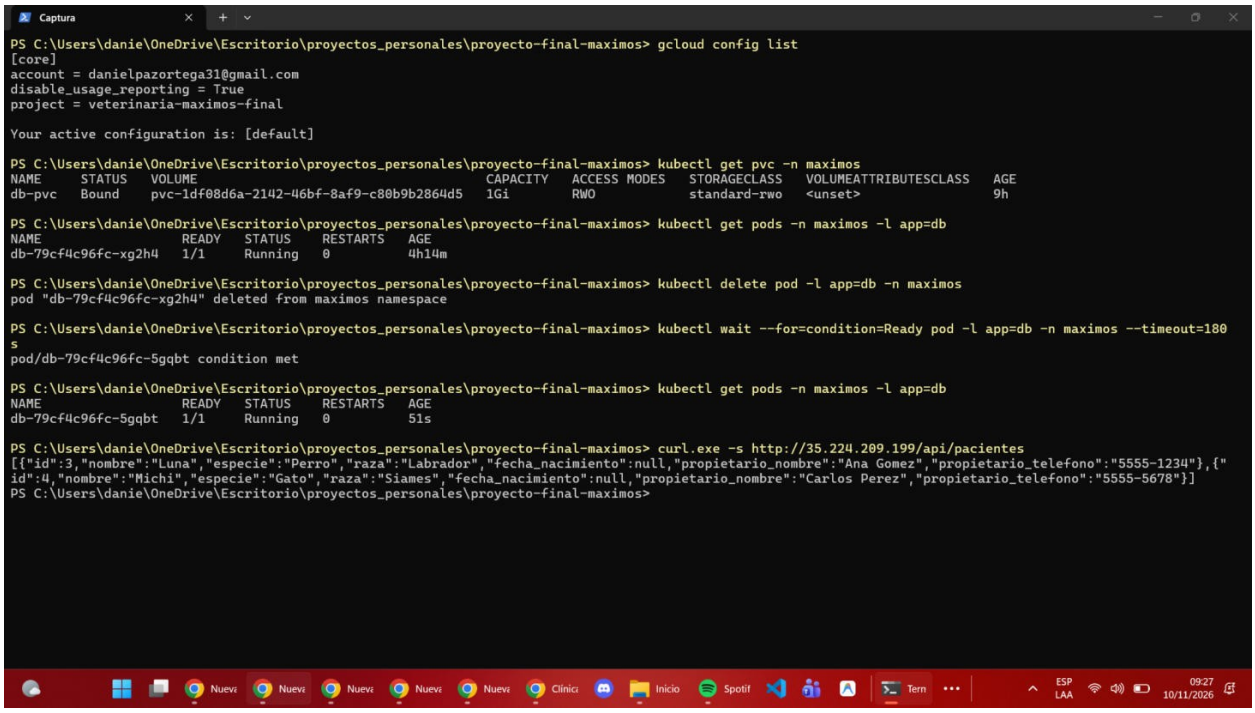
E2. ConfigMap en un Pod (P4)



E3. Secret en un Pod (P4)



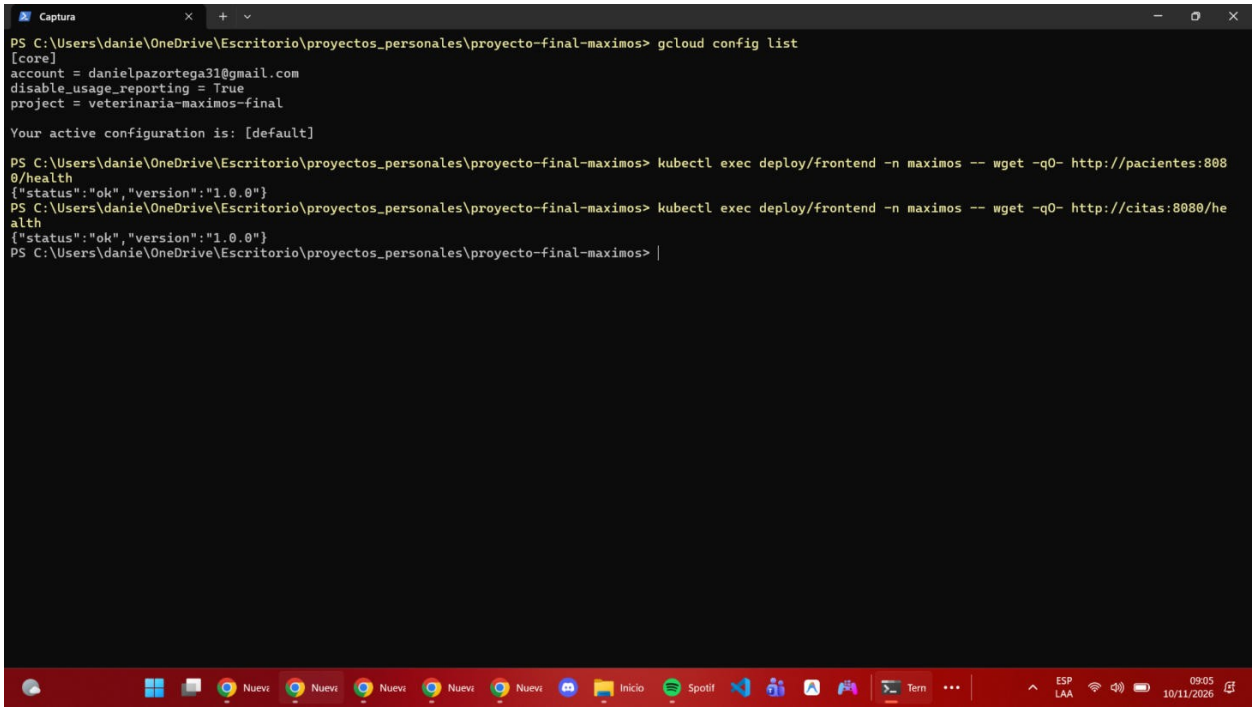
E4. Volumen persistente (P4)



E5. IP pública desde el celular (P6)

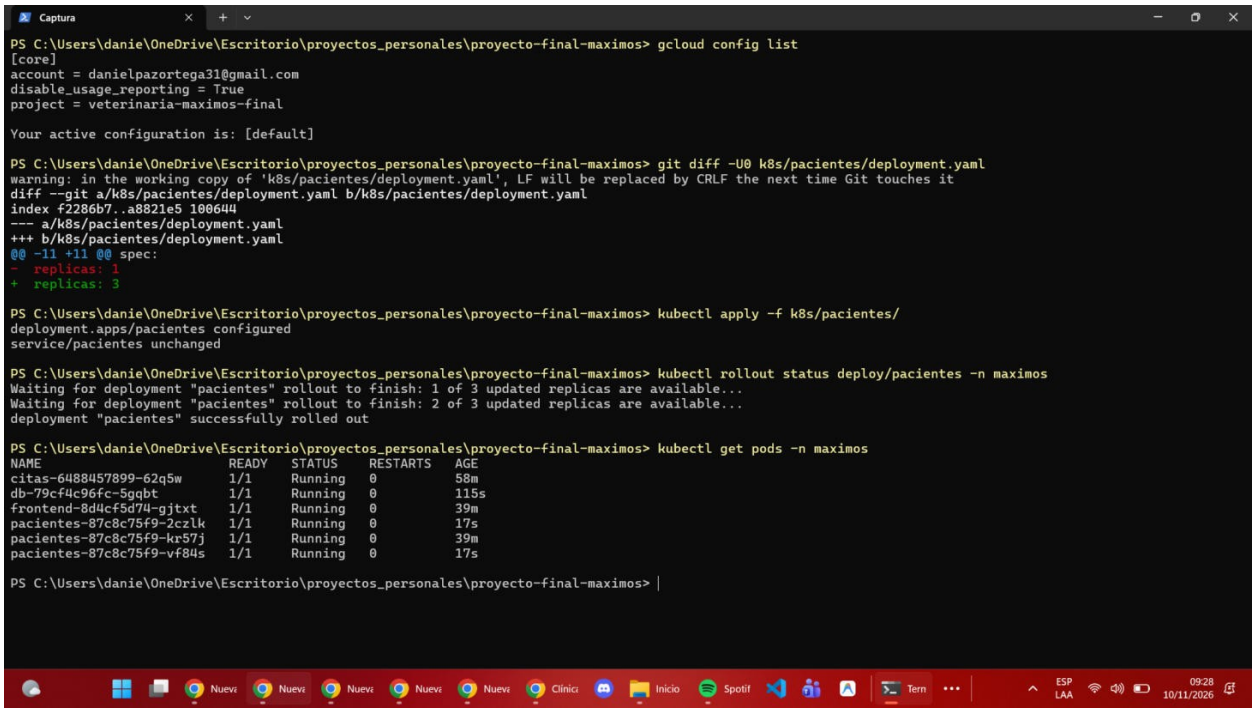


E6. Tráfico por DNS (P6)

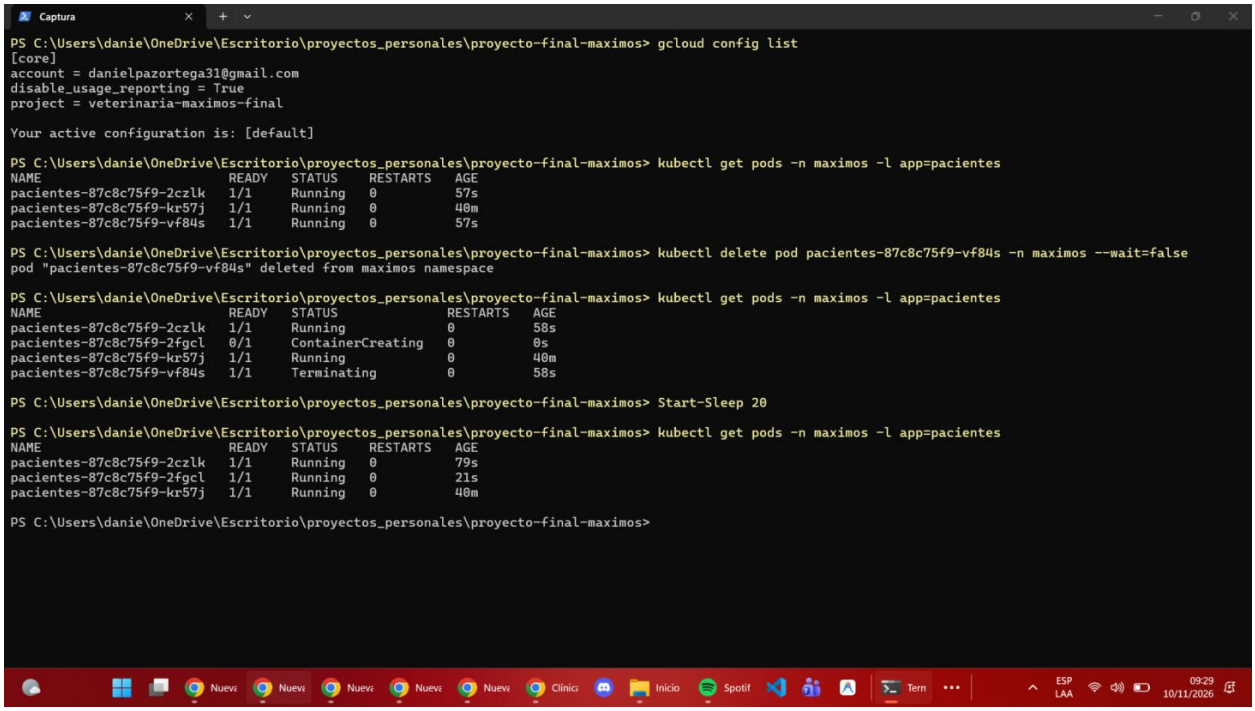


F. Operación (10 pts)

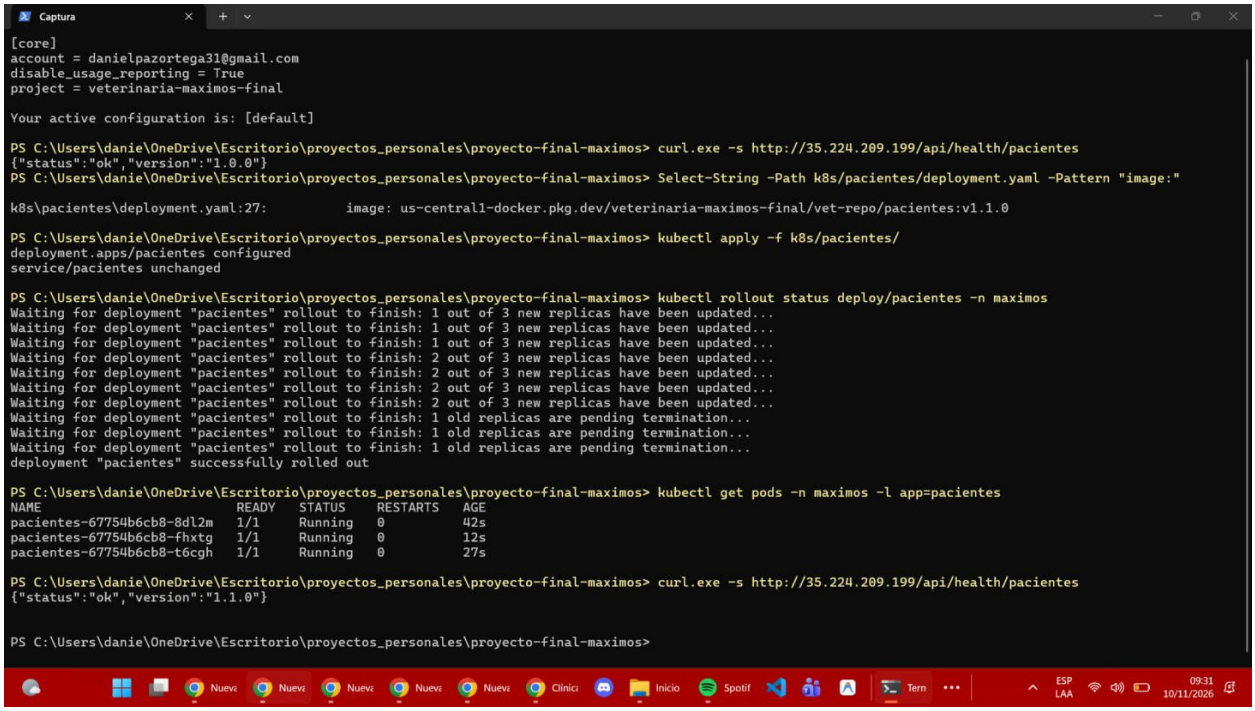
F1. Tres réplicas (P6)



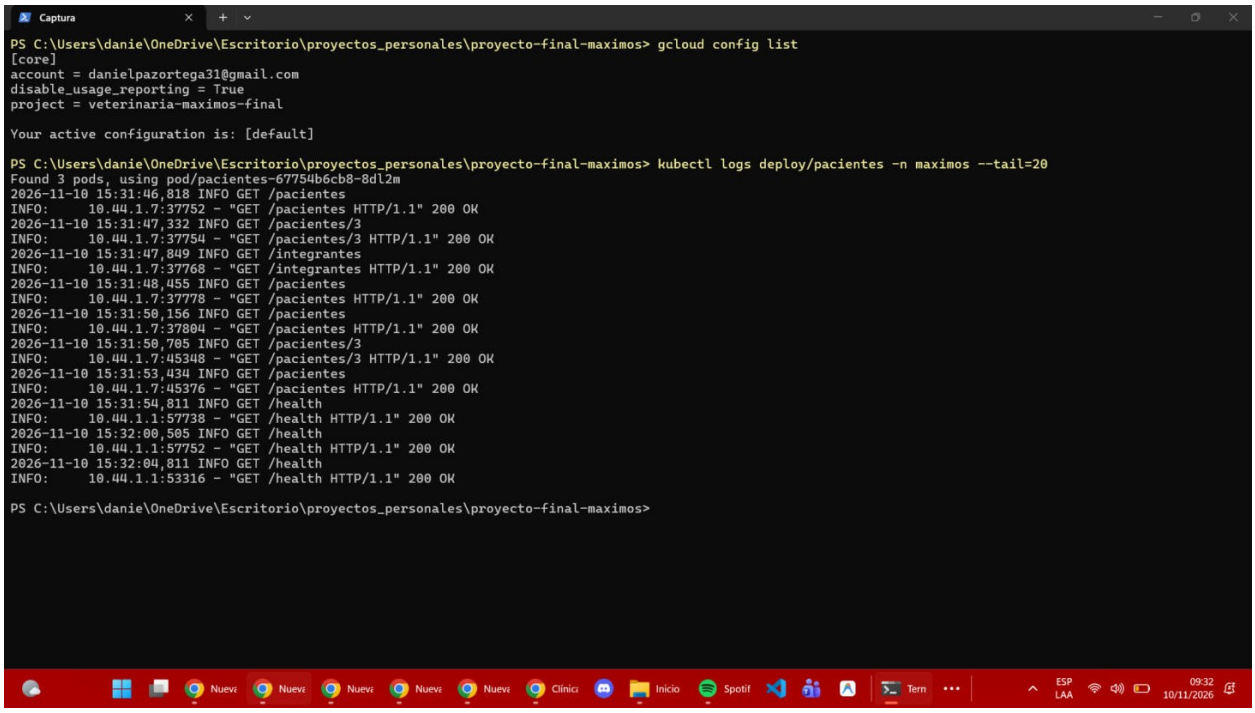
F2. Pod borrado (P6)



F3. Rolling update (P6)

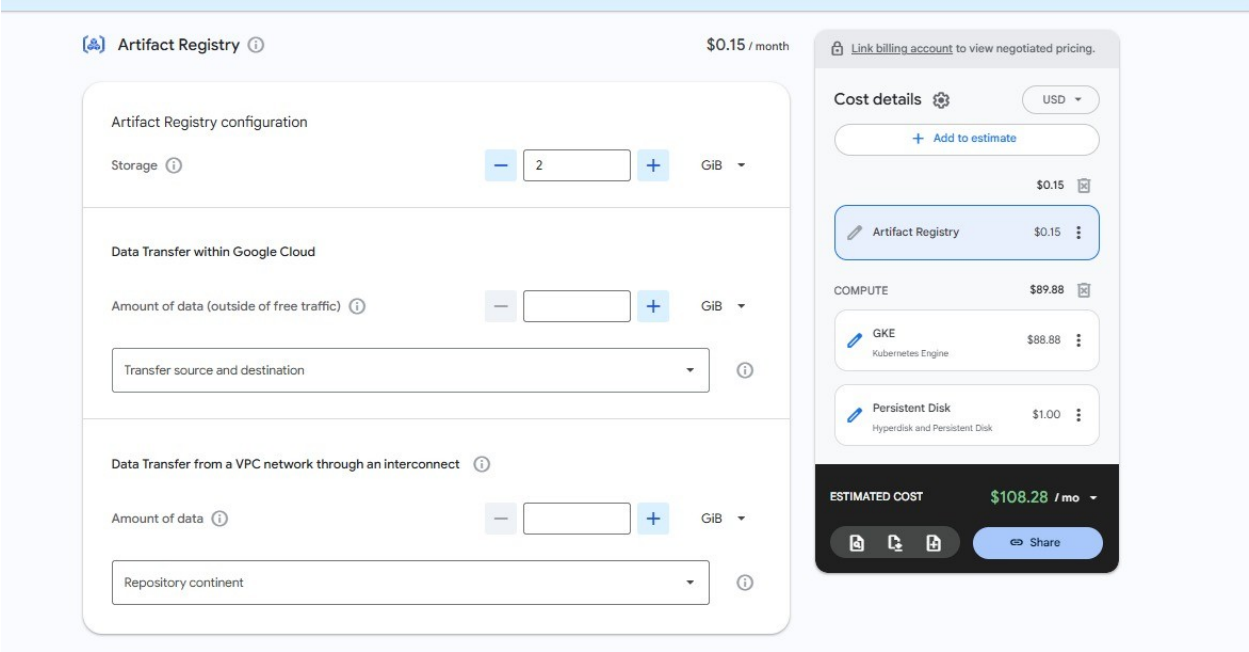


F4. Logs (P6)



G. Costos y cierre (8 pts)

G1. Estimación de costo (P5)



G3. Facturación (P5)

